

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

PENGANTAR MIKROKONTROLER

Alif Aryo Pradipto - 5024241086

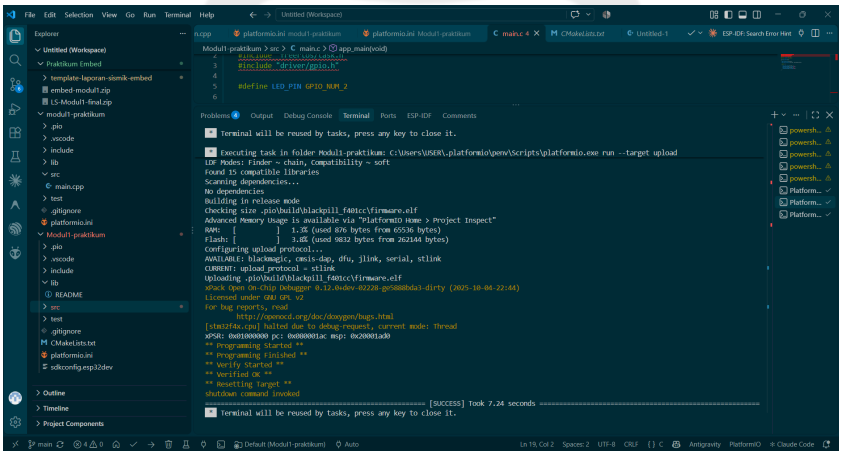
2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Percobaan pertama bertujuan untuk mengenali board STM32 Blackpill serta memahami proses pemrograman mikrokontroler menggunakan framework Arduino melalui PlatformIO pada Visual Studio Code. Adapun langkah-langkah yang dilakukan adalah sebagai berikut.

1. Mempersiapkan project Modul 1 yang telah dibuat sebelumnya pada PlatformIO, kemudian mengatur konfigurasi `platformio.ini` menggunakan board STM32 Blackpill dengan framework Arduino.
2. Menghubungkan board STM32 Blackpill dengan ST-Link menggunakan kabel jumper. Koneksi dilakukan dengan mencocokkan pin SWDIO, SWCLK, GND, dan 3.3V pada kedua perangkat.
3. Menghubungkan ST-Link ke komputer melalui kabel USB untuk memungkinkan proses pemrograman dan flashing program ke dalam mikrokontroler.
4. Menuliskan kode program Blink yang telah disediakan pada modul ke dalam file `src/main.cpp`.
5. Melakukan proses Build untuk memeriksa program, kemudian melakukan Upload agar program dapat dijalankan pada board STM32 Blackpill.



Gambar 1: percobaan 1 upload berhasill

6. Mengamati LED onboard pada pin PC13 untuk memastikan apakah LED dapat menyala dan mati secara bergantian dengan interval waktu 500 ms.

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

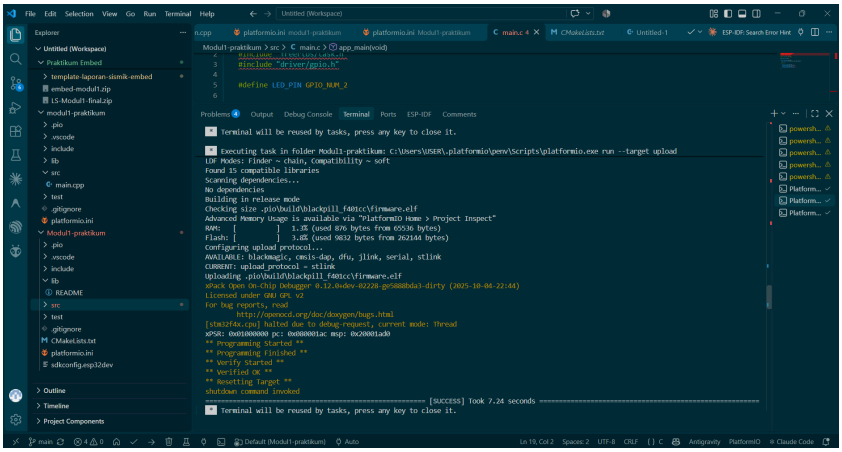
Percobaan kedua bertujuan untuk mengenali board ESP32 serta memahami penggunaan framework ESP-IDF dalam pemrograman mikrokontroler. Langkah-langkah yang dilakukan adalah sebagai berikut.

1. Mempersiapkan project Modul 1 yang telah digunakan sebelumnya, kemudian mengubah konfigurasi `platformio.ini` menggunakan board ESP32 DevKit dengan framework ESP-IDF.
2. Menghubungkan board ESP32 ke komputer menggunakan kabel USB dan memastikan perangkat telah terdeteksi oleh komputer.

3. Menghapus file `src/main.cpp` yang digunakan pada percobaan sebelumnya, kemudian membuat file baru bernama `src/main.c`.

4. Menuliskan kode program Blink menggunakan framework ESP-IDF sesuai dengan instruksi pada modul.

5. Melakukan proses Build dan Upload untuk mengompilasi serta mengirimkan program ke board ESP32.



Gambar 2: percobaan 2 build berhasill

6. Melakukan pemeriksaan terhadap koneksi dan susunan wiring pada rangkaian yang digunakan. Namun, proses percobaan mengalami kendala akibat kesalahan pemasangan kabel (wiring), sehingga program tidak dapat berjalan sebagaimana mestinya.

7. Melakukan identifikasi terhadap kemungkinan kesalahan koneksi sebagai upaya untuk mengetahui penyebab tidak berhasilnya percobaan.

1.3 Percobaan 3: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

Percobaan kedua bertujuan untuk mengenali board ESP32 serta memahami penggunaan framework ESP-IDF dalam pemrograman mikrokontroler. Langkah-langkah yang dilakukan adalah sebagai berikut.

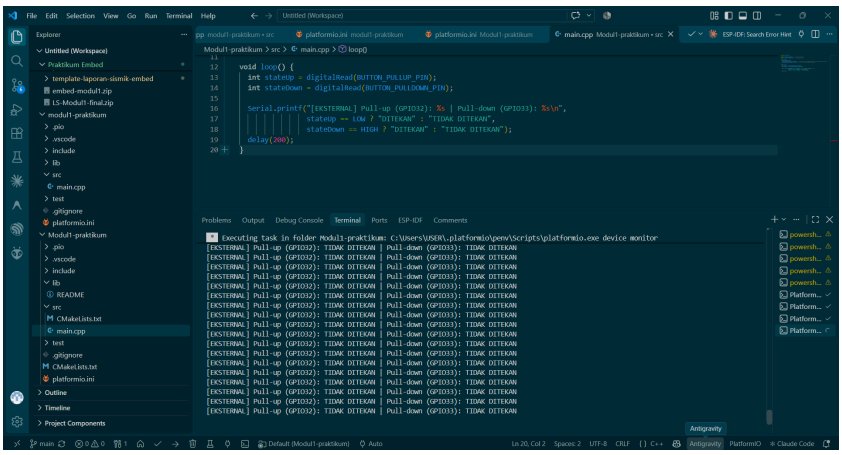
1. Mempersiapkan project Modul 1 yang telah digunakan sebelumnya, kemudian mengubah konfigurasi `platformio.ini` menggunakan board ESP32 DevKit dengan framework ESP-IDF.

2. Menghubungkan board ESP32 ke komputer menggunakan kabel USB dan memastikan perangkat telah terdeteksi oleh komputer.

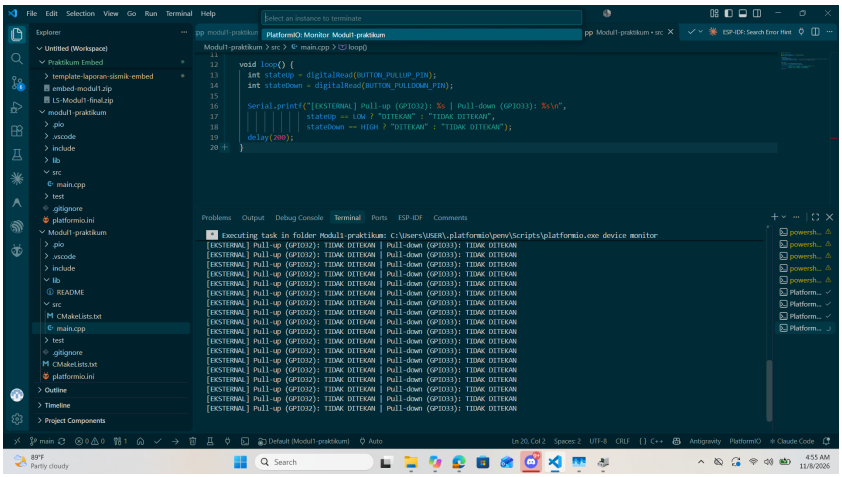
3. Menghapus file `src/main.c` yang digunakan pada percobaan sebelumnya, kemudian membuat file baru bernama `src/main.cpp`.

4. Menuliskan kode program Blink menggunakan framework ESP-IDF sesuai dengan instruksi pada modul.

5. Melakukan proses Build dan Upload untuk mengompilasi serta mengirimkan program ke board ESP32.



Gambar 3: percobaan 3A build berhasill



Gambar 4: percobaan 3B build berhasill

6. Melakukan pemeriksaan terhadap koneksi dan susunan wiring pada rangkaian yang digunakan. Namun, proses percobaan mengalami kendala akibat kesalahan pemasangan kabel (wiring), sehingga program tidak dapat berjalan sebagaimana mestinya.

7. Melakukan identifikasi terhadap kemungkinan kesalahan koneksi sebagai upaya untuk mengetahui penyebab tidak berhasilnya percobaan.

1.4 percobaan 4 dan 5

Praktikum Modul 1 pada kelompok ini hanya dapat dilaksanakan hingga Percobaan 2. Percobaan selanjutnya, yaitu Percobaan 3 mengenai konfigurasi pull-up dan pull-down, Percobaan 4 mengenai debouncing, serta Percobaan 5 mengenai pembacaan ADC menggunakan LDR, tidak dapat dilaksanakan karena keterbatasan waktu praktikum.

2 Analisis Hasil Percobaan

Praktikum Modul 1 memperkenalkan penggunaan mikrokontroler melalui dua board yang berbeda, yaitu STM32 Blackpill dan ESP32. Kedua board tersebut memiliki karakteristik serta lingkungan pengembangan yang berbeda. Dalam pelaksanaannya, kelompok melakukan Percobaan 1 menggu-

nakan STM32 Blackpill dengan framework Arduino dan Percobaan 2 menggunakan ESP32 dengan framework ESP-IDF.

Pada Percobaan 1, pengujian dilakukan menggunakan board STM32 Blackpill yang diprogram melalui PlatformIO dengan framework Arduino. Proses pemrograman dilakukan dengan bantuan ST-Link melalui protokol komunikasi SWD. Program Blink menggunakan fungsi `setup()` untuk mengatur pin PC13 sebagai output dan fungsi `loop()` untuk menjalankan perintah menyalakan serta mematikan LED secara berulang. Berdasarkan pelaksanaan percobaan, program dapat dijalankan pada board STM32 Blackpill sehingga LED onboard dapat berkedip dengan interval 500 ms. Hasil tersebut menunjukkan bahwa proses konfigurasi project, koneksi ST-Link, serta proses flashing program telah berjalan dengan baik.

Pada Percobaan 2, pengujian dilanjutkan menggunakan board ESP32 dengan framework ESP-IDF. Berbeda dengan framework Arduino yang menggunakan fungsi `setup()` dan `loop()`, ESP-IDF menggunakan fungsi `app_main()` sebagai titik awal program. Pengaturan GPIO dilakukan secara eksplisit menggunakan fungsi `gpio_reset_pin()`, `gpio_set_direction()`, dan `gpio_set_level()`. Selain itu, fungsi `vTaskDelay()` digunakan sebagai mekanisme penundaan berbasis FreeRTOS.

Namun, pada pelaksanaan Percobaan 2, program Blink belum berhasil dijalankan sebagaimana yang diharapkan. Kendala yang ditemukan berkaitan dengan kesalahan pemasangan kabel (wiring) pada rangkaian. Kesalahan tersebut menyebabkan koneksi antarperangkat tidak sesuai dengan konfigurasi yang diperlukan, sehingga proses pengujian tidak dapat menghasilkan keluaran yang diharapkan. Kondisi ini menunjukkan bahwa keberhasilan pemrograman mikrokontroler tidak hanya ditentukan oleh ketepatan kode program, tetapi juga bergantung pada ketepatan koneksi perangkat keras.

Secara teoritis, ESP32 dapat menjalankan program Blink melalui LED onboard yang terhubung pada GPIO 2. Program seharusnya mengatur pin tersebut sebagai output, kemudian memberikan logika HIGH dan LOW secara bergantian dengan jeda 500 ms. Akan tetapi, karena terdapat kendala pada pemasangan wiring, hasil tersebut tidak dapat diverifikasi melalui pengamatan langsung selama praktikum. Oleh karena itu, keberhasilan program pada Percobaan 2 belum dapat dipastikan.

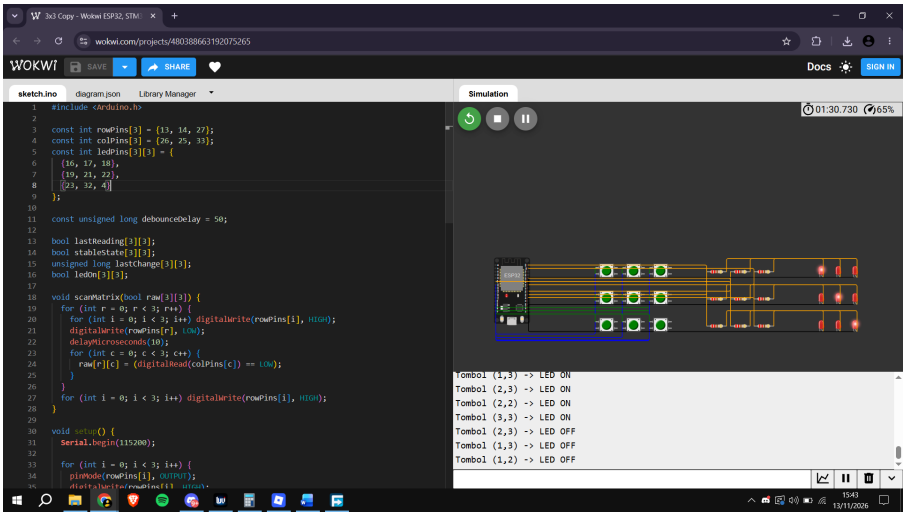
Adapun Percobaan 4, dan Percobaan 5 tidak dilaksanakan karena keterbatasan waktu praktikum. Dengan demikian, kelompok belum memperoleh hasil pengamatan langsung terkait konfigurasi pull-up dan pull-down resistor, penerapan debouncing pada input tombol, maupun pembacaan nilai analog menggunakan ADC dan sensor LDR. Ketiga percobaan tersebut hanya dapat dipahami melalui materi dan dasar teori yang terdapat pada modul.

Berdasarkan pelaksanaan praktikum, dapat diketahui bahwa ketelitian dalam melakukan konfigurasi perangkat keras merupakan salah satu faktor penting dalam pengembangan sistem tertanam. Kesalahan sederhana dalam menghubungkan pin dapat menghambat proses pengujian meskipun kode program telah disusun sesuai dengan instruksi. Selain itu, pengelolaan waktu selama praktikum juga berpengaruh terhadap jumlah percobaan yang dapat diselesaikan.

3 Hasil Tugas Modul

Bagian ini memuat hasil pengerjaan tugas tambahan yang diberikan pada Modul 1, yaitu simulasi Matrix Pushbutton 3x3 yang digunakan untuk mengendalikan Matrix LED 3x3.

3.1 Hasil Simulasi



Gambar 5: Hasil Simulasi Matrix Pushbutton 3x3 dan Matrix LED 3x3

3.2 Source Code

Listing 1: Program Matrix Pushbutton 3x3 dan Matrix LED 3x3

```
#include <Arduino.h>

const int rowPins[3] = {13, 14, 27};
const int colPins[3] = {26, 25, 33};

const int ledPins[3][3] = {
    {16, 17, 18},
    {19, 21, 22},
    {23, 32, 4}
};

const unsigned long debounceDelay = 50;

bool lastReading[3][3];
bool stableState[3][3];
unsigned long lastChange[3][3];
bool ledOn[3][3];

void scanMatrix(bool raw[3][3]) {
    for (int r = 0; r < 3; r++) {
        for (int i = 0; i < 3; i++)
            digitalWrite(rowPins[i], HIGH);

        digitalWrite(rowPins[r], LOW);
        delayMicroseconds(10);

        for (int c = 0; c < 3; c++) {
            raw[r][c] = (digitalRead(colPins[c]) == LOW);
        }
    }

```

```
    for (int i = 0; i < 3; i++)
        digitalWrite(rowPins[i], HIGH);
}

void setup() {
    Serial.begin(115200);

    for (int i = 0; i < 3; i++) {
        pinMode(rowPins[i], OUTPUT);
        digitalWrite(rowPins[i], HIGH);
        pinMode(colPins[i], INPUT_PULLUP);
    }

    for (int r = 0; r < 3; r++) {
        for (int c = 0; c < 3; c++) {
            pinMode(ledPins[r][c], OUTPUT);
            digitalWrite(ledPins[r][c], LOW);

            lastReading[r][c] = false;
            stableState[r][c] = false;
            lastChange[r][c] = 0;
            ledOn[r][c] = false;
        }
    }

    Serial.println("Matrix Pushbutton 3x3 -> Matrix LED 3x3");
}

void loop() {
    bool raw[3][3];
    scanMatrix(raw);
    unsigned long now = millis();

    for (int r = 0; r < 3; r++) {
        for (int c = 0; c < 3; c++) {

            if (raw[r][c] != lastReading[r][c]) {
                lastChange[r][c] = now;
                lastReading[r][c] = raw[r][c];
            }

            if ((now - lastChange[r][c]) > debounceDelay &&
                raw[r][c] != stableState[r][c]) {

                stableState[r][c] = raw[r][c];

                if (stableState[r][c]) {
                    ledOn[r][c] = !ledOn[r][c];

                    digitalWrite(
                        ledPins[r][c],
                        ledOn[r][c] ? HIGH : LOW
                    );

```

```
86                      Serial.printf(
87                          "Tombol (%d,%d) -> LED %s\n",
88                          r + 1, c + 1,
89                          ledOn[r][c] ? "ON" : "OFF"
90                      );
91                  }
92              }
93          }
94      }
95  }
```

3.3 Pertanyaan Analisis

1. Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing pada C.5 Debouncing! Jawab:

   Debouncing tetap dibutuhkan karena fenomena bouncing tidak dipengaruhi oleh jumlah tombol yang ditekan. Bouncing terjadi akibat getaran mekanis pada kontak logam di dalam tombol ketika berada dalam kondisi terhubung maupun terputus. Oleh karena itu, meskipun hanya satu tombol yang ditekan, tetap dapat muncul beberapa perubahan kondisi HIGH dan LOW dalam waktu yang sangat singkat.

   Pada rangkaian matrix pushbutton, program melakukan pemindaian (scanning) terhadap seluruh baris dan kolom secara berulang dengan interval waktu yang cepat. Hal ini menyebabkan setiap perubahan sinyal akibat bouncing berpotensi terdeteksi sebagai input penekanan yang berbeda. Karena LED menggunakan mekanisme toggle, jumlah pembacaan yang tidak sesuai dapat menyebabkan LED kembali ke kondisi awal atau menghasilkan perubahan kondisi yang tidak konsisten. Dengan menerapkan debouncing pada setiap tombol, perubahan status hanya dilakukan setelah sinyal berada dalam kondisi stabil selama 50 ms. Dengan demikian, satu kali penekanan tombol dapat dikenali sebagai satu input saja.

2. Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan floating? Jawab:

   Pada rangkaian ini digunakan konfigurasi internal pull-up (INPUT-PULLUP) pada ketiga pin kolom. Konfigurasi tersebut menyebabkan pin kolom memiliki kondisi HIGH ketika tidak ada tombol yang ditekan karena terhubung secara internal ke tegangan 3,3 V. Sebaliknya, ketika tombol pada baris yang sedang diaktifkan (LOW) ditekan, pin kolom akan terhubung ke baris tersebut sehingga menghasilkan pembacaan LOW.

   Sementara itu, baris yang tidak sedang digunakan diatur dalam kondisi high-impedance melalui mode INPUT. Pengaturan ini bertujuan untuk menghindari kemungkinan terjadinya hubungan singkat antarbaris ketika beberapa tombol pada kolom yang sama ditekan secara bersamaan. Kondisi floating perlu dihindari karena tegangan pada pin menjadi tidak pasti dan rentan terhadap gangguan sinyal (noise), baik dari proses pemindaian maupun pengaruh lingkungan sekitar. Akibatnya, sistem dapat mendeteksi perubahan logika secara keliru dan menganggap terdapat tombol yang ditekan meskipun sebenarnya tidak ada input dari pengguna.

3. Andaikan salah satu tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai digital input (tombol, hanya HIGH/LOW) dan sebagai analog input (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?

   Jawab:

   Pembacaan digital input bekerja dengan membandingkan tegangan pada pin terhadap batas logika tertentu, sehingga hanya menghasilkan dua kemungkinan kondisi, yaitu HIGH atau LOW. Metode ini sesuai untuk pushbutton karena tombol hanya memiliki dua keadaan, yaitu ditekan dan tidak ditekan.

   Berbeda dengan tombol, LDR memiliki karakteristik resistansi yang berubah secara bertahap mengikuti intensitas cahaya yang diterima. Perubahan resistansi tersebut menyebabkan tegangan pada rangkaian pembagi tegangan ikut berubah secara kontinu. Melalui ADC 12-bit pada ESP32, perubahan tegangan ini dikonversi menjadi nilai digital dengan rentang 0 hingga 4095, sehingga dapat merepresentasikan 4096 tingkat pembacaan yang berbeda dengan resolusi sekitar 0,8 mV per level.

   Hal tersebut memungkinkan sistem membedakan berbagai tingkat intensitas cahaya, sebagaimana ditunjukkan pada Percobaan 5, ketika nilai ADC berada di sekitar 357 saat LDR tertutup dan meningkat hingga sekitar 3049 ketika terkena cahaya. Berbeda dengan pushbutton, LDR tidak dapat langsung dimasukkan ke dalam proses scanning matriks karena membutuhkan pembacaan analog. Oleh sebab itu, LDR perlu dihubungkan ke pin ADC tersendiri, misalnya GPIO 34. Apabila LDR ingin digunakan sebagai pengganti tombol, program dapat menetapkan nilai ambang (threshold) tertentu, misalnya LED akan menyala ketika hasil pembacaan ADC kurang dari 1000.

4 Kesimpulan

Berdasarkan pelaksanaan Praktikum Modul 1 mengenai Pengantar Mikrokontroler, dapat disimpulkan bahwa PlatformIO pada Visual Studio Code dapat digunakan sebagai lingkungan pengembangan untuk memprogram berbagai jenis mikrokontroler dengan framework yang berbeda. Pada Percobaan 1, penggunaan STM32 Blackpill dengan framework Arduino memberikan pemahaman mengenai konfigurasi GPIO sebagai output digital serta proses flashing program menggunakan ST-Link melalui protokol SWD. Program Blink berhasil dijalankan sehingga LED onboard dapat menyala dan mati secara bergantian.

Pada Percobaan 2, penggunaan ESP32 dengan framework ESP-IDF memperkenalkan struktur pemrograman yang berbeda dari framework Arduino. ESP-IDF menggunakan fungsi `app_main()` sebagai titik awal program serta menyediakan pengaturan GPIO secara lebih eksplisit melalui fungsi-fungsi yang disediakan oleh driver GPIO. Namun, percobaan ini belum berhasil diselesaikan karena terdapat kesalahan pada pemasangan wiring, sehingga keluaran program tidak dapat diamati sesuai dengan tujuan pengujian.

Selain itu, Percobaan 4 hingga Percobaan 5 tidak dapat dilaksanakan akibat keterbatasan waktu praktikum. Oleh karena itu, kelompok belum dapat melakukan debouncing pada input tombol, serta pembacaan sinyal analog menggunakan ADC dan LDR.

Secara keseluruhan, praktikum ini memberikan pemahaman awal mengenai penggunaan STM32 Blackpill dan ESP32, konfigurasi lingkungan pengembangan, serta pentingnya kesesuaian antara perangkat keras dan perangkat lunak dalam sistem tertanam. Praktikum ini juga menunjukkan bahwa ketelitian dalam melakukan wiring dan pengelolaan waktu merupakan faktor penting untuk menunjang keberhasilan setiap tahapan percobaan.

5 Lampiran

5.1 Dokumentasi saat Praktikum

Foto dokumentasi kegiatan praktikum tidak tersedia. Bukti pelaksanaan percobaan berupa tangkapan layar pada bagian Langkah-Langkah Percobaan dan Hasil Tugas Modul.